DocFlow

Take-Home Coding Exercise

Build a multi-client document processing platform with AI-powered classification and extraction, stage-based workflows, and human review.

Prepared for **Greg Berns**

gregberns@gmail.com

Estimated time: 3–4 days

Tech stack: your choice

For reference, our stack is Java 25 / Spring Boot 4, React / TypeScript, PostgreSQL

DOCFLOW-BEE81E57A1C78 DOCFLOW-BEE81E57A1C78 DOCFLOW-BEE81E57A1C78

Overview

You're building **DocFlow**, a document processing platform for an outsourced bookkeeping firm. The firm processes financial documents for multiple business clients. Each client has different document types, different data that needs to be extracted from those documents, and different processing workflows.

Your application should support the three clients described in this document, with support for additional clients as needed.

A note on tools: *You're welcome to use any tools you'd like, including AI assistants. We care about the result and your ability to explain it — expect follow-up questions about your design decisions, tradeoffs, and implementation details.*

How It Works

1. A user selects which client (organization) they're working on.

2. They upload a document (PDF or image).

3. The system **classifies** the document — determines what type it is (invoice, receipt, etc.) using an external service (e.g., an LLM API).

4. The system **extracts** data from the document based on its type — pulling out the relevant fields using an external service (e.g., an LLM API).

5. The document enters a **Review** stage where a human reviews the extracted data, confirms or corrects the document type, and edits any fields that were extracted incorrectly.

6. After review, the document moves through one or more **approval stages** (which vary by client and document type) before being marked as **Filed**.

Documents can also be **Rejected** at the Review stage (terminal — the document is done) or **Flagged** at any approval stage (sent back to Review with a comment).

Requirements

General

- The application must be runnable with `docker-compose up`. All services (backend, frontend, database, etc.) should start with this single command.
- You may use any programming language, framework, and database you prefer.
- Classification and data extraction must use a real external service (e.g., OpenAI, Anthropic, Google, or any other LLM/AI API). Do not hardcode or simulate extraction results.
- Sample documents are provided in the `samples/` directory for testing.

Dashboard

- The first screen is an **organization selector**. The user picks which client they want to work on. Everything from that point is scoped to that client.
- The main screen is a **document list** showing all documents for the selected organization.
- Documents can be filtered by **stage** and **document type**. The available stages and types in the filter dropdowns should reflect the selected organization's document types and workflows.

- Documents that are still being classified or extracted should appear at the top of the list with a visual indicator that they are not yet actionable.
- Clicking a document opens its **detail view**.

Detail View

The detail view has two panels:

- **Left:** A preview of the uploaded document.
- **Right:** Document information, a stage progress indicator, and either a form (Review) or a read-only summary (approval stages and Filed), with action buttons at the bottom.

The **stage progress indicator** shows the document's position in its workflow — which stages are complete, which is current, and which are upcoming.

Review Stage

When a document is at the Review stage:

- The extracted fields are displayed in an **editable form**. The fields shown depend on the document type and the client (see client specs below).
- A **document type dropdown** allows the reviewer to confirm or change the classification. If the document type is changed, the system should **re-extract** data using the new type's field definitions, and alert the user that re-extraction will occur.
- **Actions: Approve** (advances to the next stage in the workflow) and **Reject** (terminal — document is done).

Review Stage (Flagged)

When a document has been flagged back to Review from an approval stage:

- A **comment banner** is displayed at the top showing the flag comment and which stage it was sent back from.
- The **Approve** button is replaced by a **Resolve** button. Hitting Resolve:
  - Checks if the document type was changed. If yes, triggers re-extraction.
  - If the document type was not changed, returns the document to the stage it was flagged from.
  - Clears the flag comment.
- The system must track which stage the document was flagged from, so it can return there after resolution.

Approval Stages

When a document is at any approval stage (e.g., Manager Approval, Attorney Approval, Partner Approval):

- The reviewed data is displayed **read-only** — the approver sees what was confirmed during review but cannot edit it.
- **Actions: Approve** (advances to the next stage) and **Flag** (sends the document back to Review).
- Clicking **Flag** opens a modal where the user must enter a comment explaining the issue. The comment is required. Submitting sends the document back to Review.

Filed / Rejected

Terminal states. The detail view shows read-only data and a back button. No actions available.

Client Specifications

Client A: Riverside Bistro

Restaurant chain

Document types and fields:

Document Type	Fields

Invoice	• vendor
	• invoiceNumber
	• invoiceDate
	• dueDate
	• lineItems
	  – description
	  – quantity
	  – unitPrice
	  – total
	• subtotal
	• tax
	• totalAmount
	• paymentTerms

Receipt	• merchant • date • amount • category (food, supplies, equipment, services) • paymentMethod

Expense Report	• employeeName
	• department
	• submissionDate
	• items
	  – date
	  – description
	  – amount
	  – category
	• totalAmount

Workflows:

Document Type	Stages
Invoice	Upload → Classify → Extract → Review → Manager Approval → Filed
Receipt	Upload → Classify → Extract → Review → Filed
Expense Report	Upload → Classify → Extract → Review → Manager Approval → Finance Approval → Filed

Client B: Pinnacle Legal Group

Law firm

Document types and fields:

Document Type	Fields
Invoice	• vendor • invoiceNumber • invoiceDate • matterNumber • matterName • amount • billingPeriod • paymentTerms
Retainer Agreement	• clientName • matterType • hourlyRate • retainerAmount • effectiveDate • termLength • scope

Expense Report	• attorneyName
	• matterNumber
	• submissionDate
	• items
	  – date
	  – description
	  – amount
	  – billable (yes/no)
	• totalAmount

Workflows:

Document Type	Stages
Invoice	Upload → Classify → Extract → Review → Attorney Approval → Billing Approval → Filed
Retainer Agreement	Upload → Classify → Extract → Review → Partner Approval → Filed
Expense Report	Upload → Classify → Extract → Review → Attorney Approval → Billing Approval → Filed

Client C: Ironworks Construction

General contractor

Document types and fields:

Document Type	Fields

Invoice	• vendor
	• invoiceNumber
	• invoiceDate
	• projectCode
	• projectName
	• amount
	• materials
	  – item
	  – quantity
	  – unitCost
	• deliveryDate
	• paymentTerms

Change Order	• projectCode • projectName • description • costImpact • scheduleImpact • requestedBy • approvedBy
Lien Waiver	• subcontractor • projectCode • projectName • amount • throughDate • waiverType (conditional, unconditional)

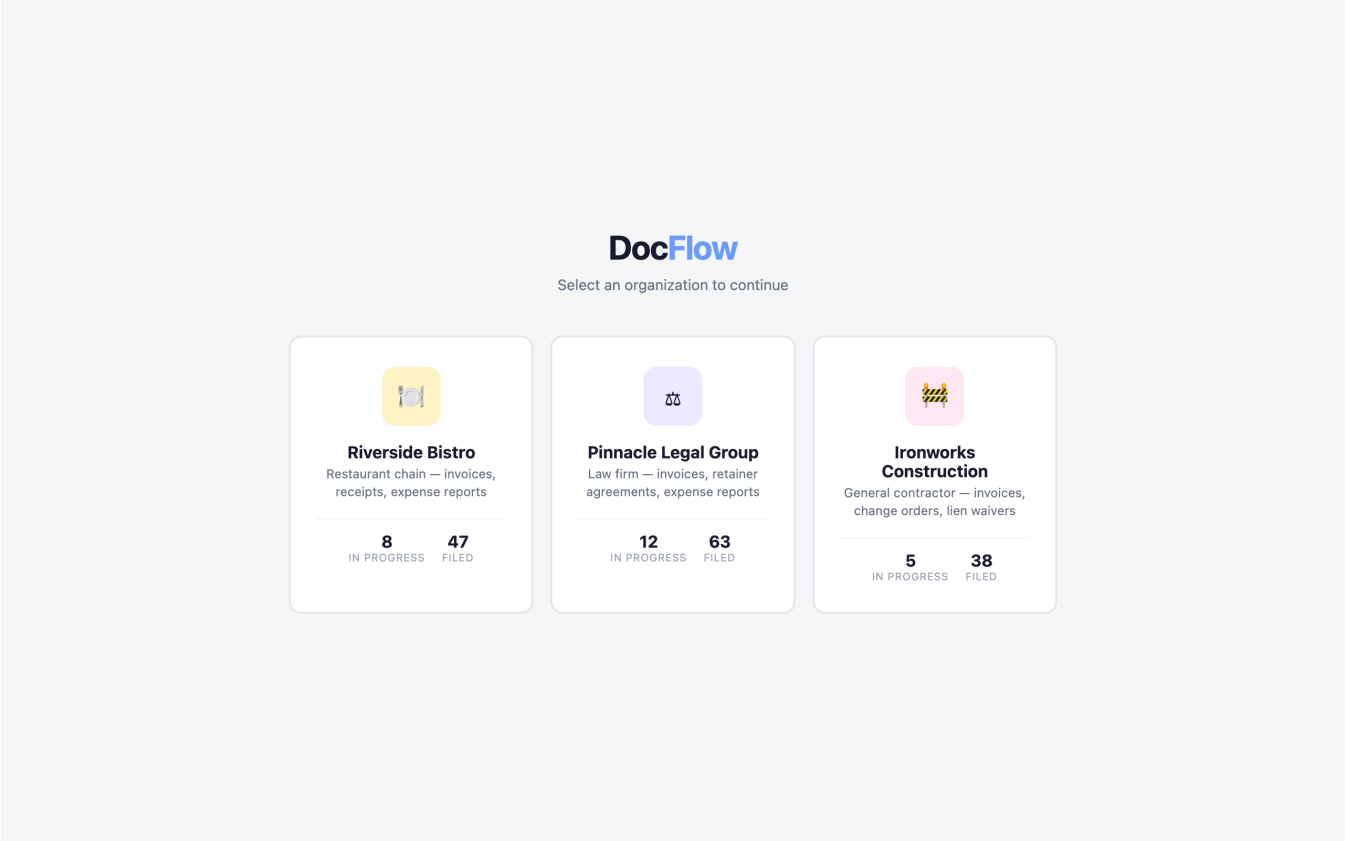
Workflows:

Document Type	Stages
Invoice	Upload → Classify → Extract → Review → Project Manager Approval → Accounting Approval → Filed
Change Order	Upload → Classify → Extract → Review → Project Manager Approval → Client Approval → Filed
Lien Waiver	Upload → Classify → Extract → Review → Project Manager Approval → Filed

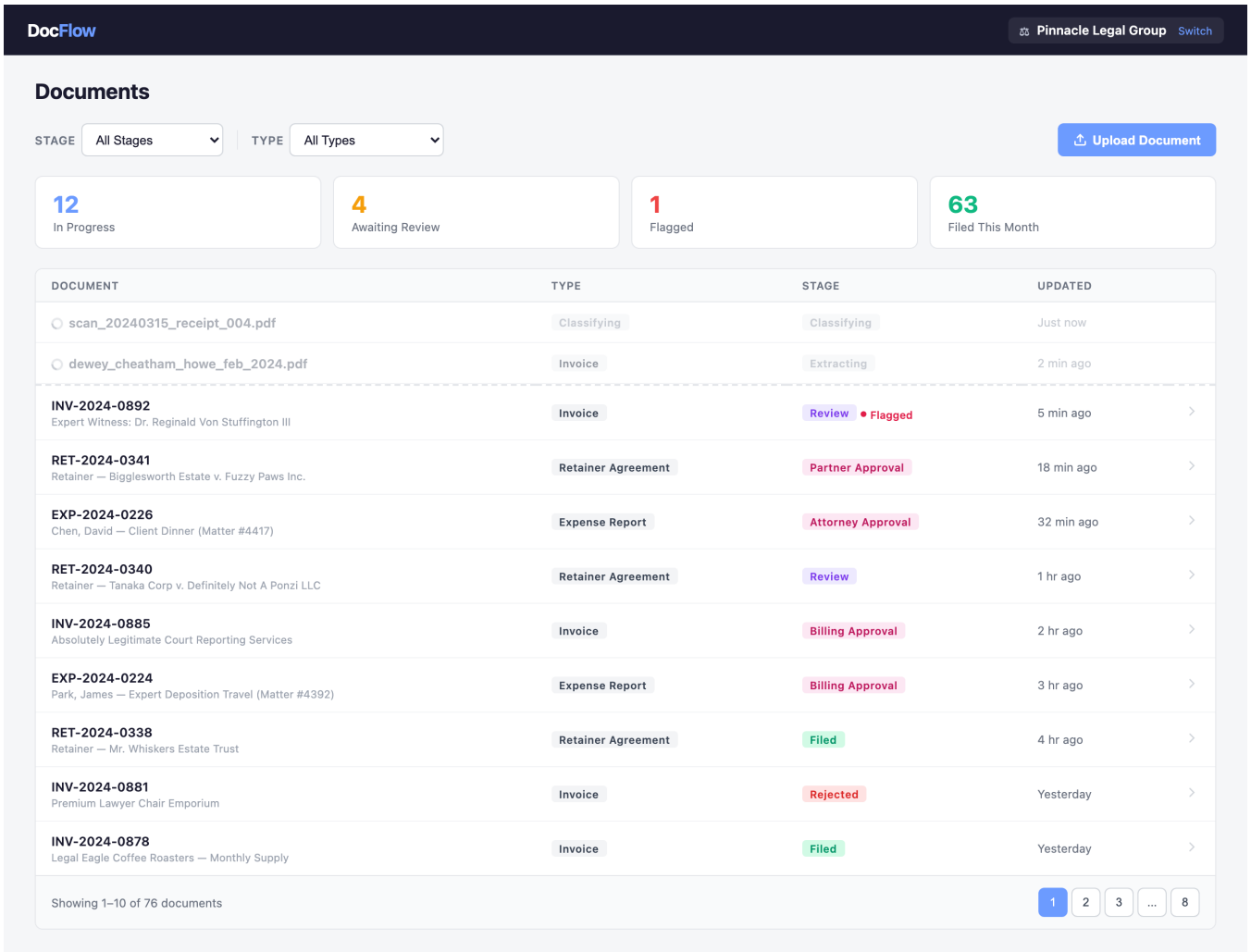
Note: *For lien waivers, if the waiver type is* ***unconditional****, the document should skip Project Manager Approval and go directly from Review to Filed.*

UI Reference

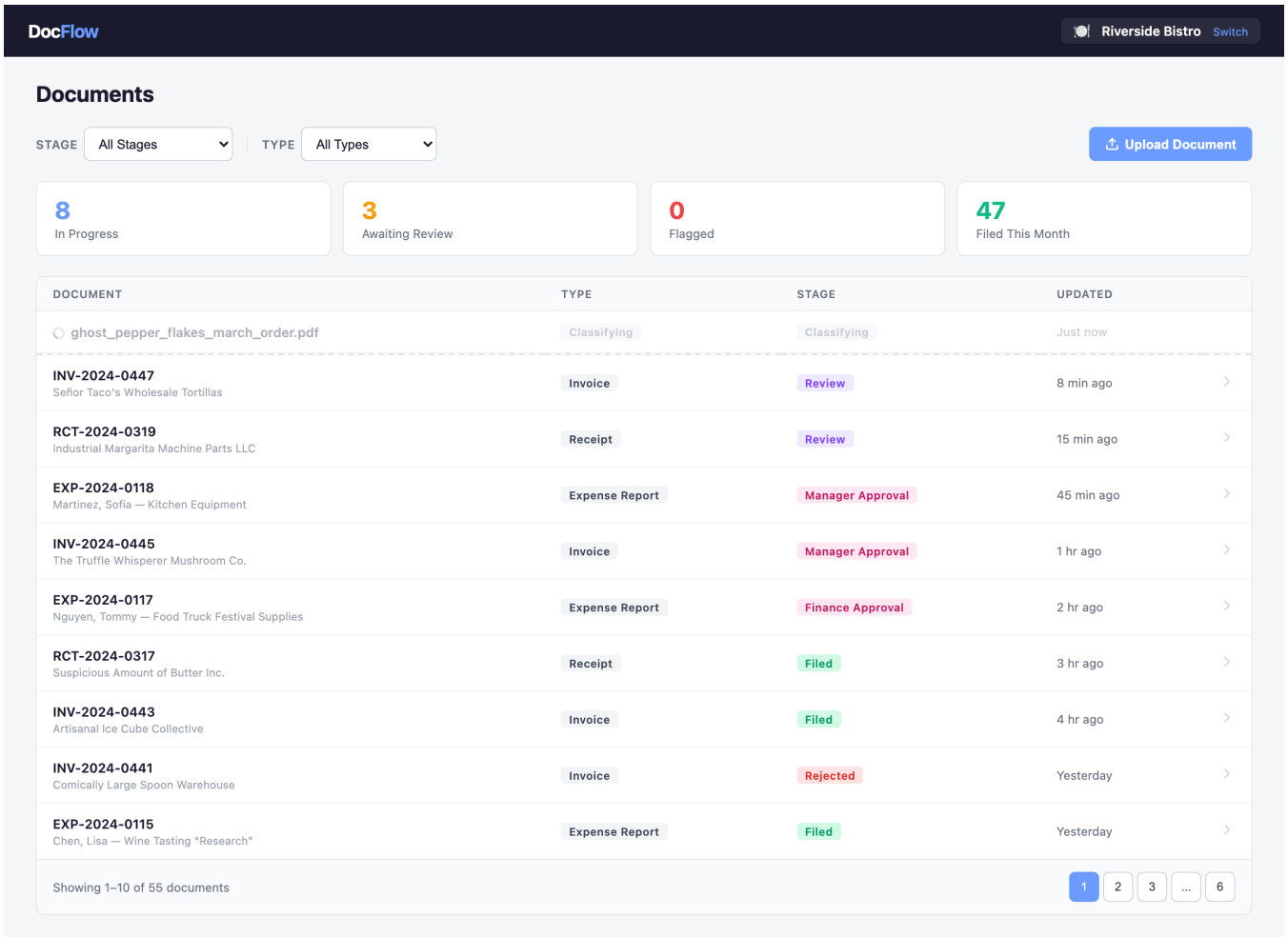
The following mockups show key screens and states. HTML versions are also provided in the `mockups/` directory — open them in a browser for full-resolution viewing. Your implementation does not need to match pixel-for-pixel, but the layout, behavior, and information shown should be consistent.



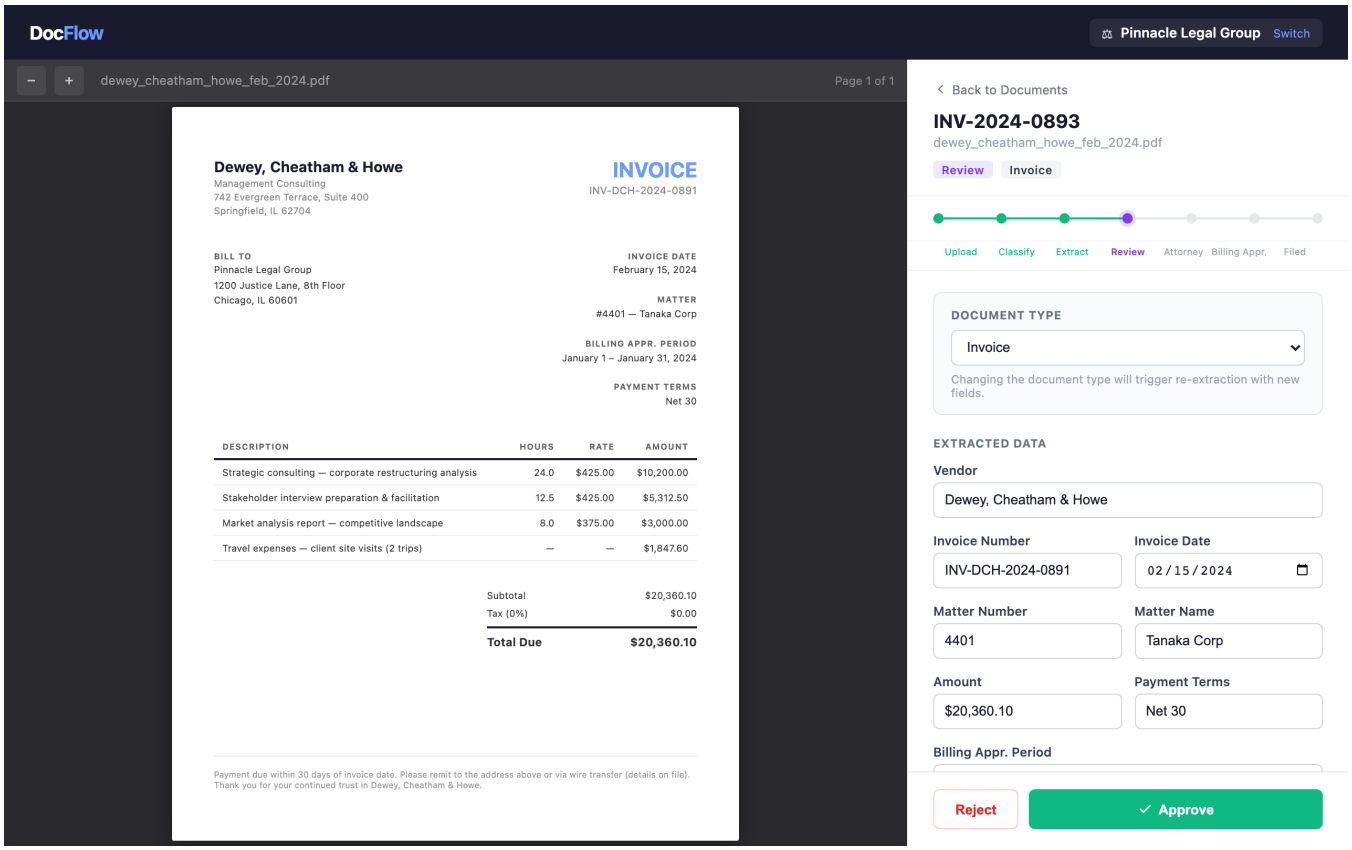
Organization Selector



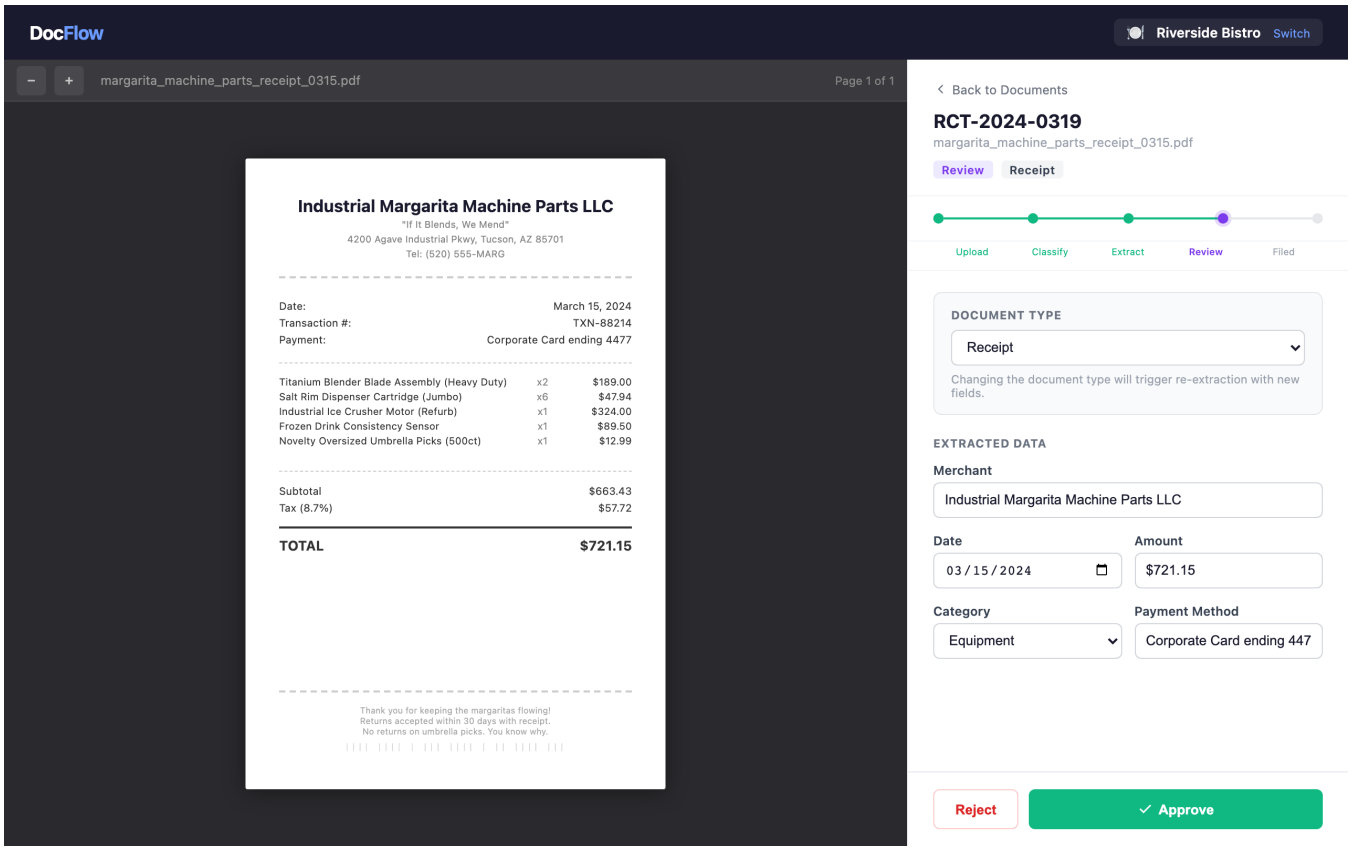
Dashboard — Pinnacle Legal Group



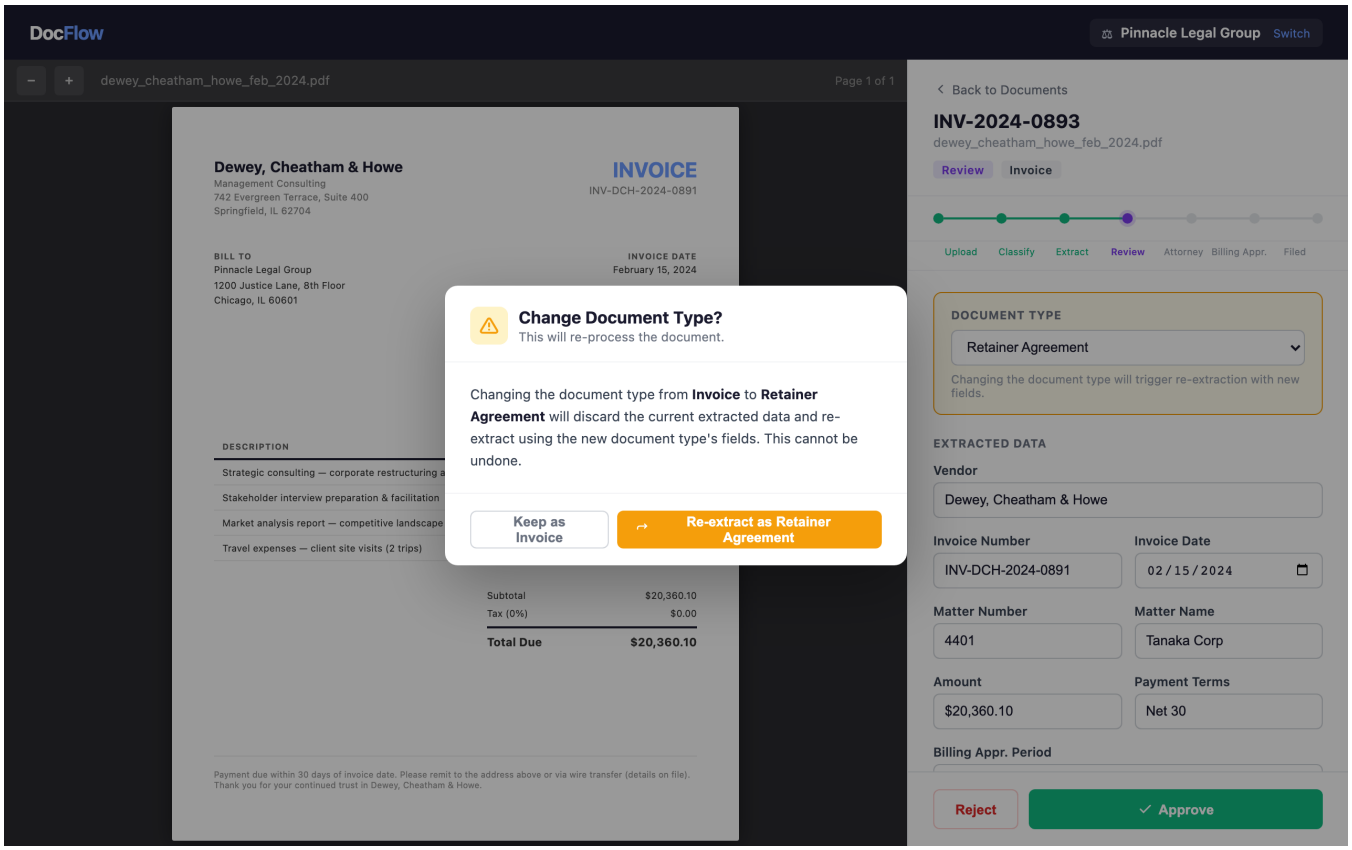
Dashboard — Riverside Bistro



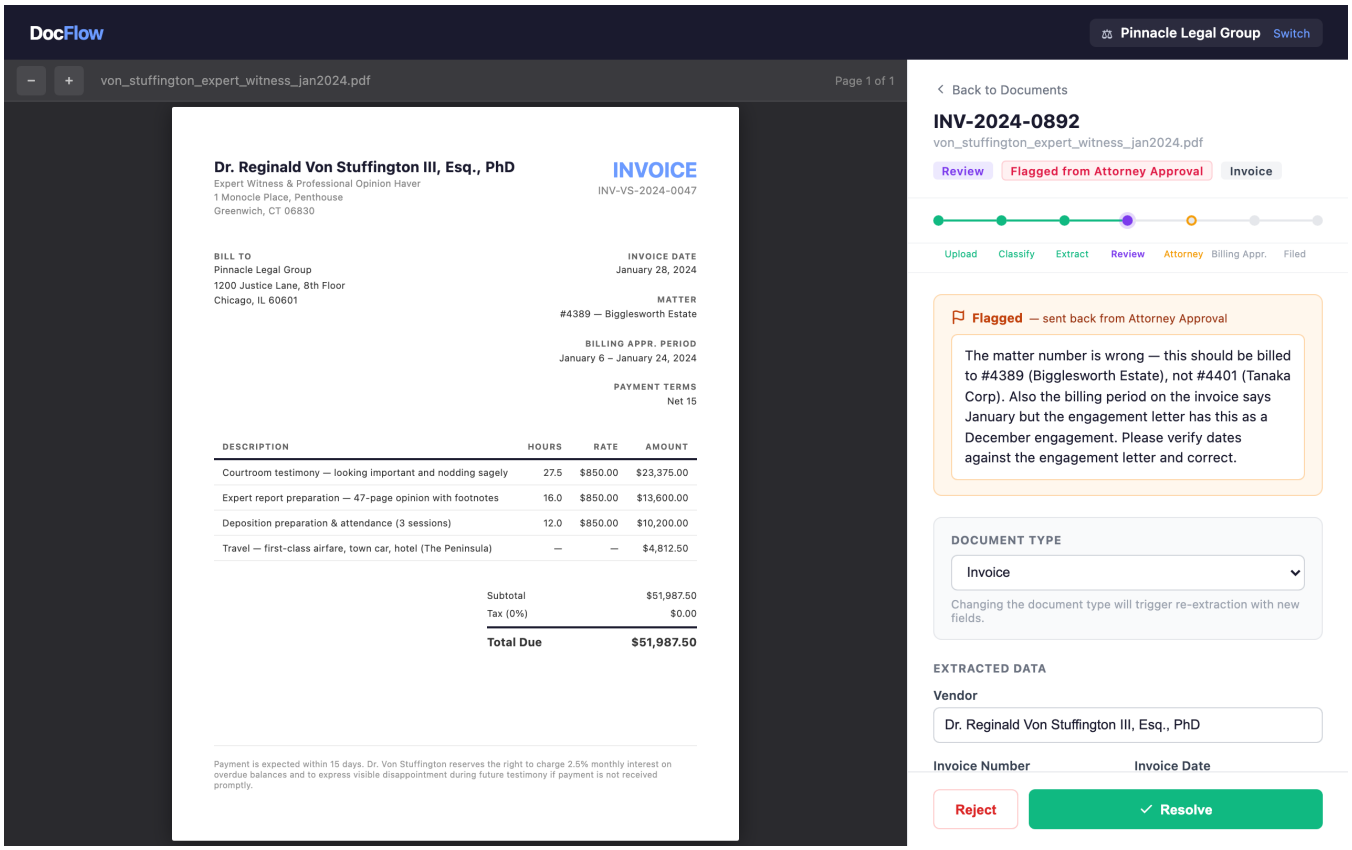
Review — Pinnacle Invoice



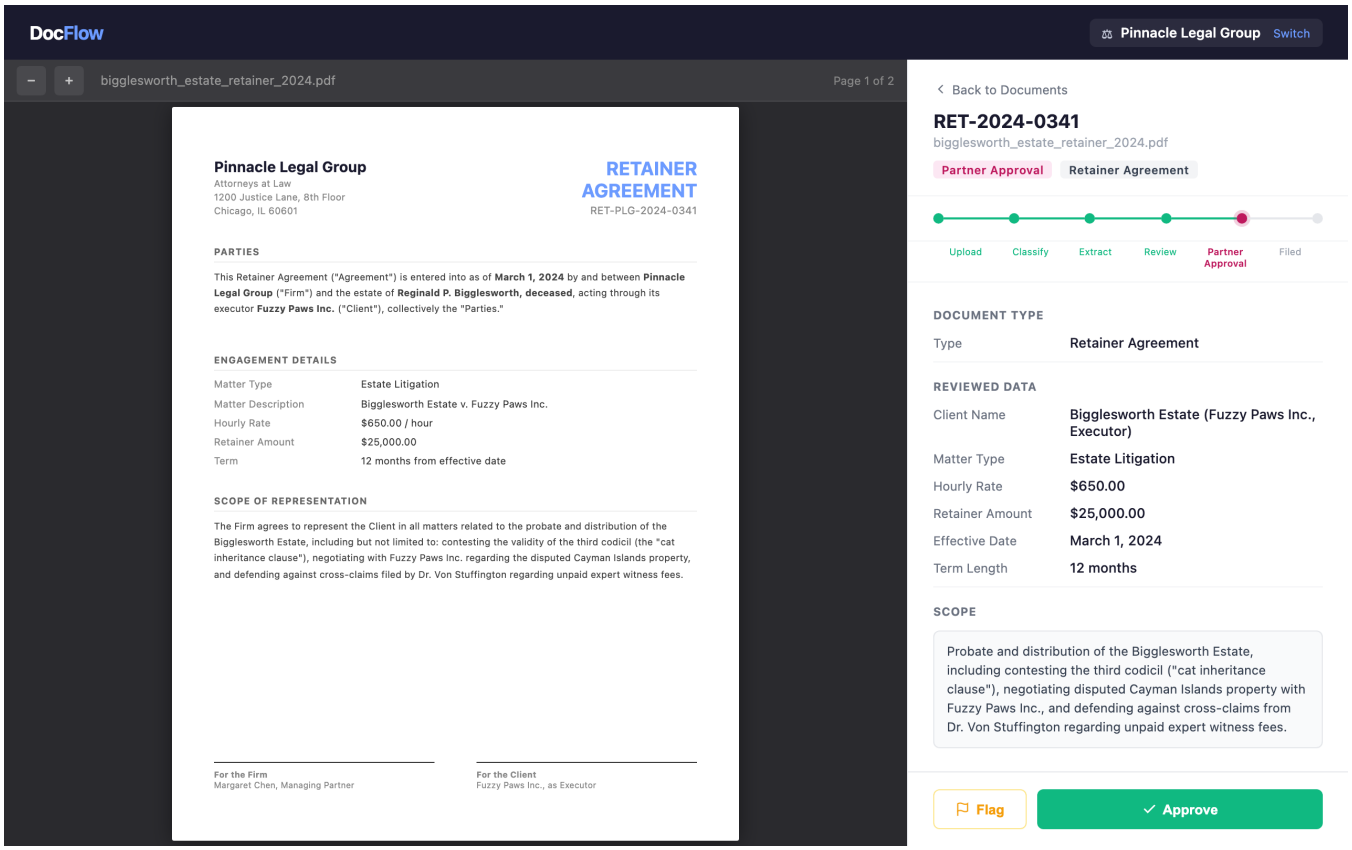
Review — Riverside Receipt



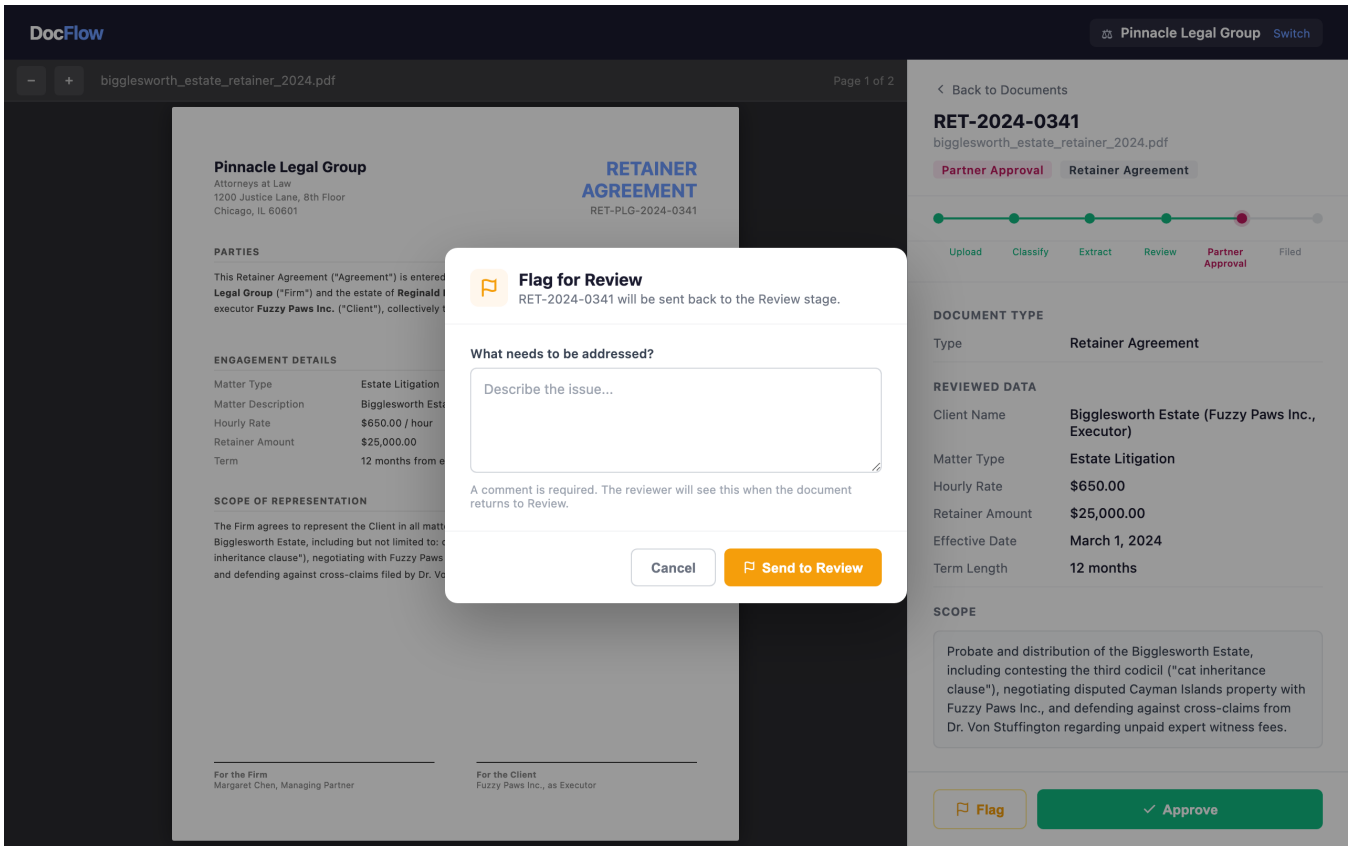
Review (Reclassify Alert) — Pinnacle Invoice



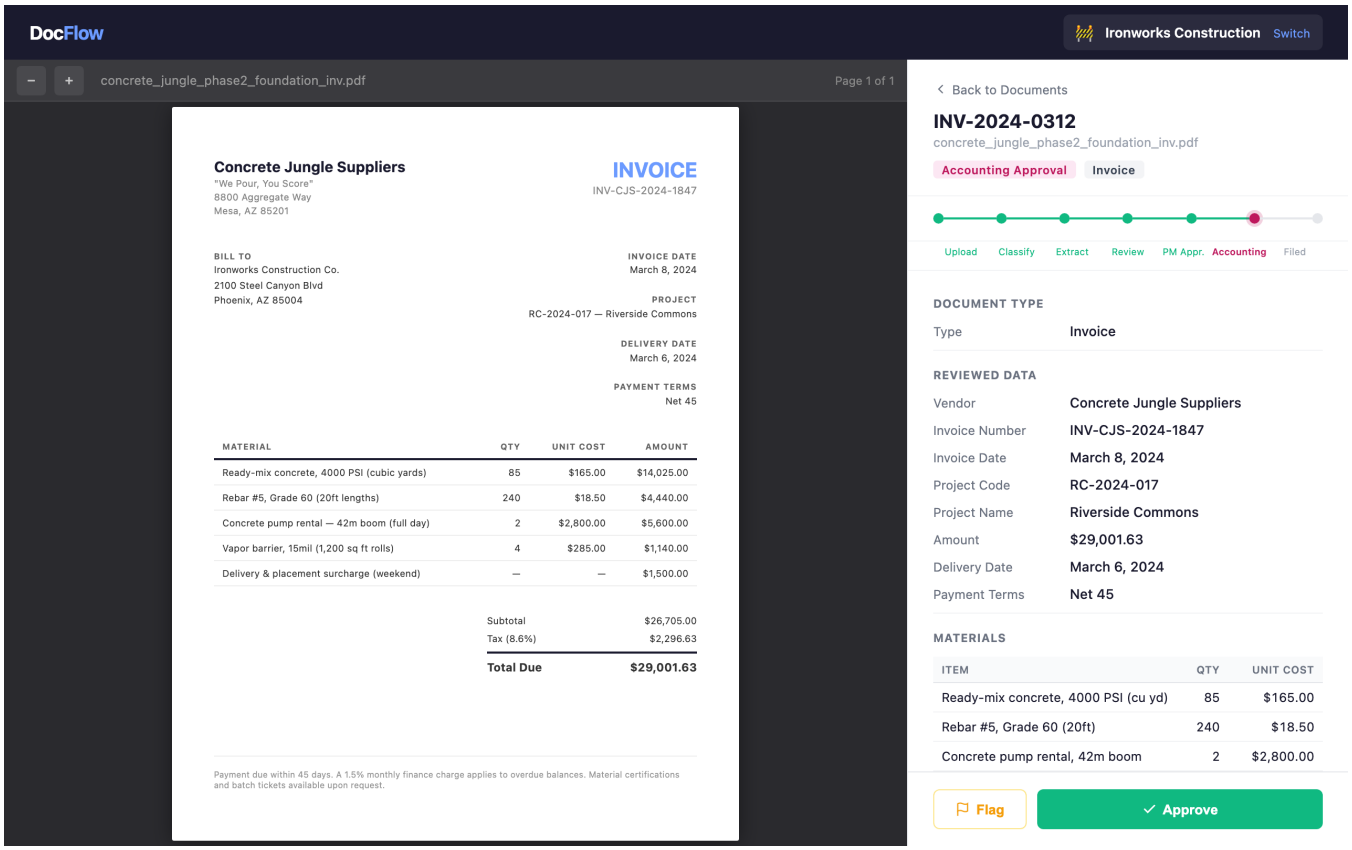
Review (Flagged) — Pinnacle Invoice



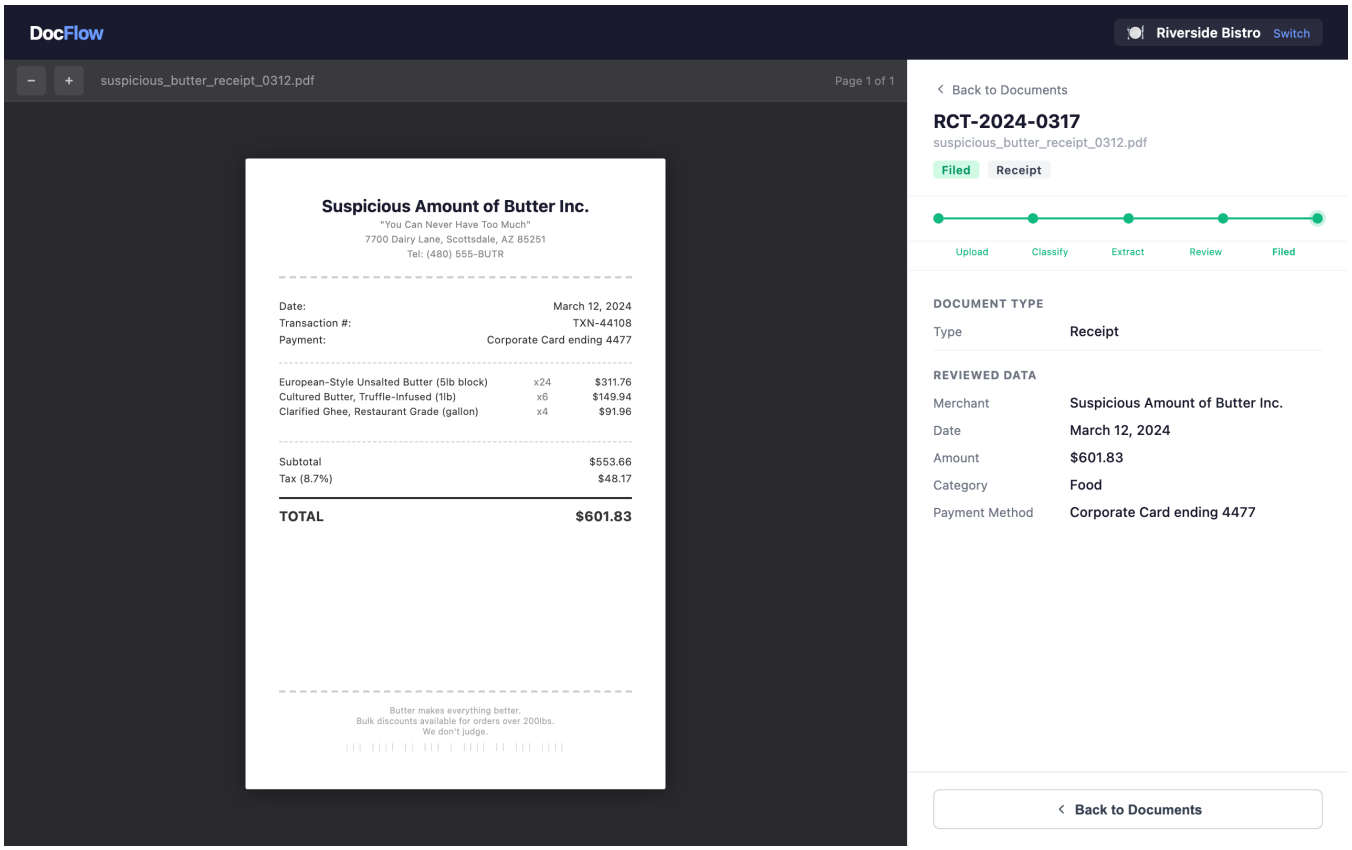
Approval — Pinnacle Retainer Agreement



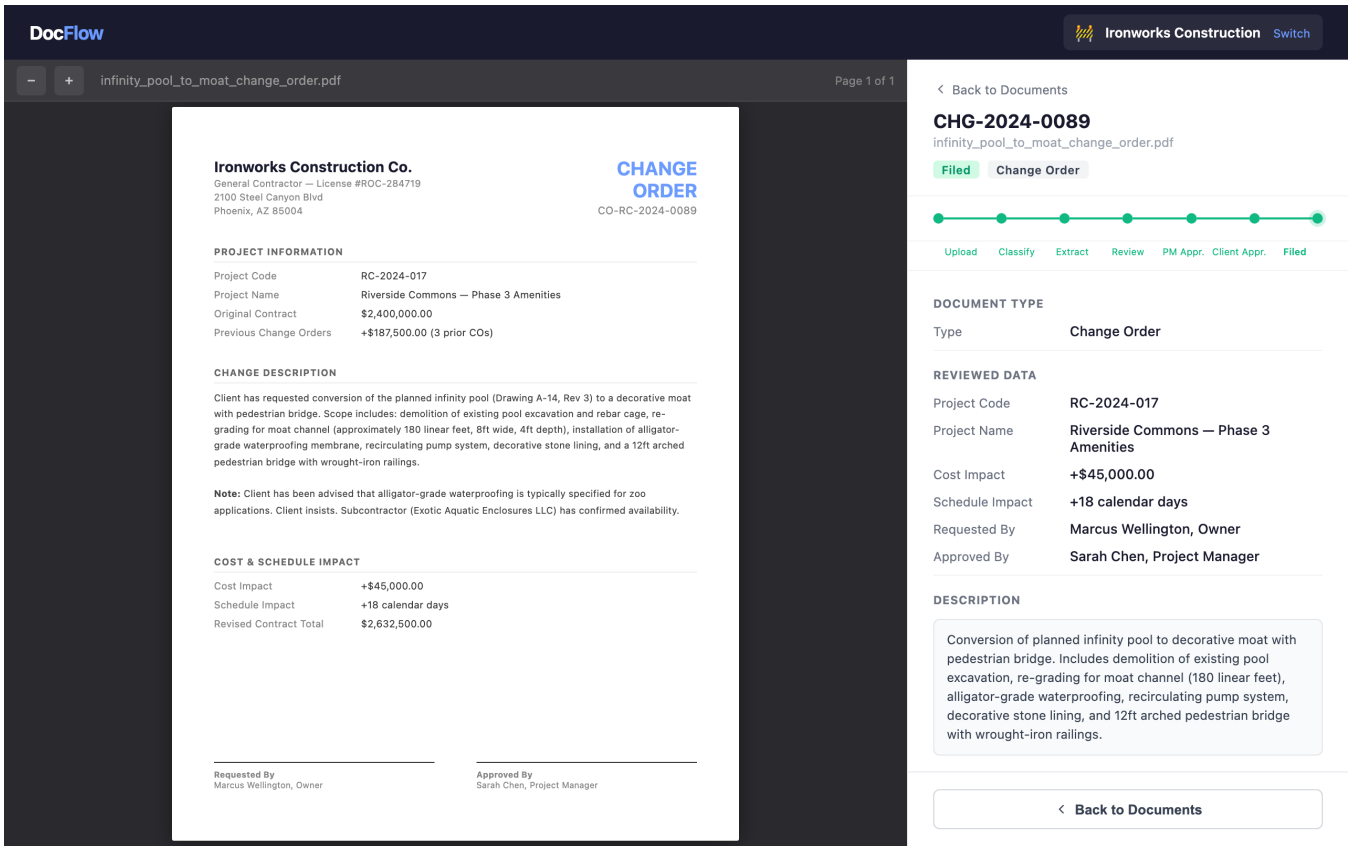
Approval — Flag Modal



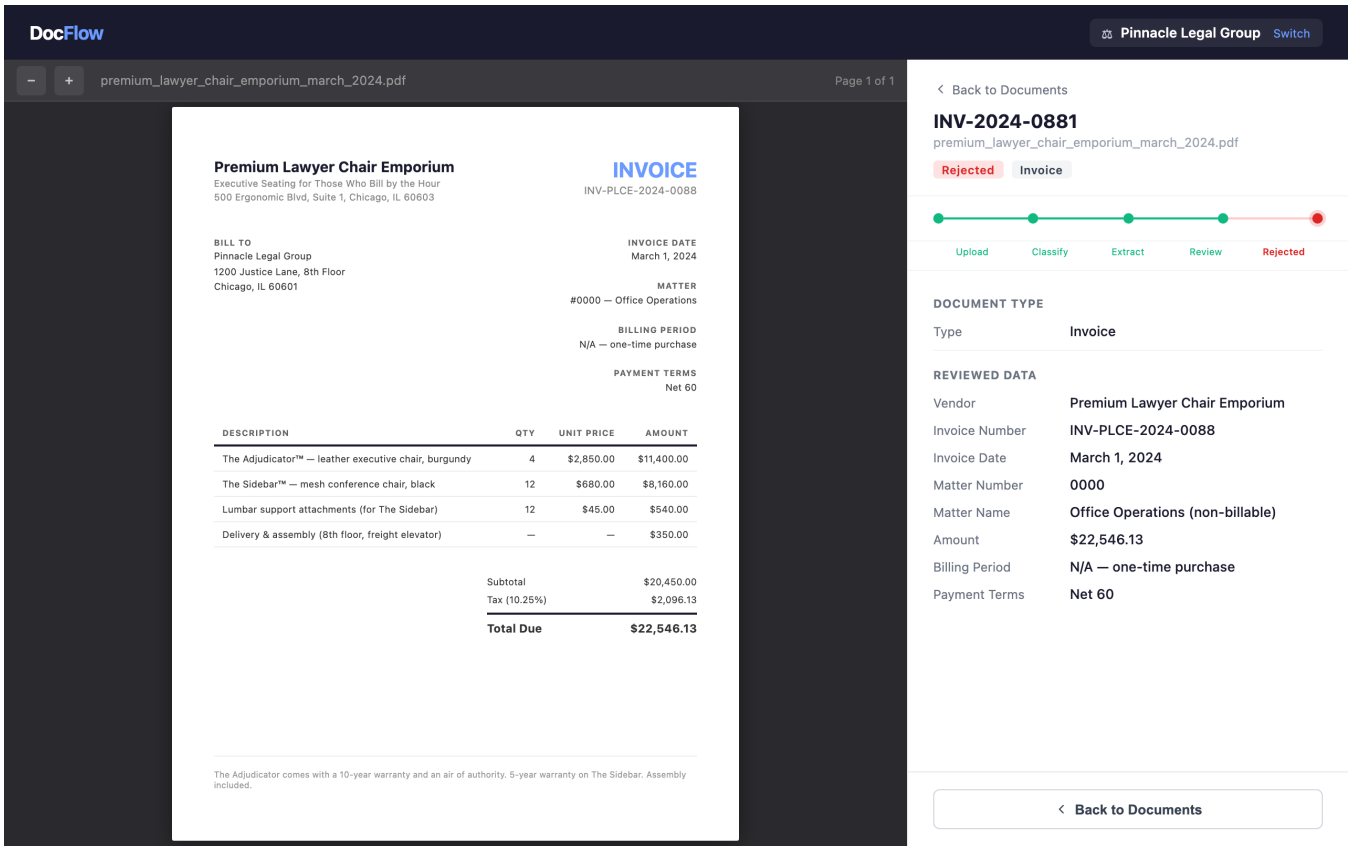
Approval — Ironworks Invoice



Filed — Riverside Receipt



Filed — Ironworks Change Order



Rejected — Pinnacle Invoice

Sample Documents

The `samples/` directory contains sample documents organized by client and document type. Use these to test your classification and extraction pipeline.

```
samples/
    riverside-bistro/
        invoices/             (3 PDFs)
        receipts/             (3 PDFs)
        expense-reports/      (2 PDFs)
    pinnacle-legal/
        invoices/             (3 PDFs)
        retainer-agreements/  (2 PDFs)
        expense-reports/      (2 PDFs)
    ironworks-construction/
        invoices/             (3 PDFs)
        change-orders/        (2 PDFs)
        lien-waivers/         (3 PDFs)
```

Submission

- Push your code to a Git repository and share the link.
- Include a brief `README.md` in your repo explaining how to run the application and any design decisions you made.
- There is no time limit, but this exercise is designed to take **3–4 days**. Most candidates report 2–3 days of actual working time — the wider window is there so you can let ideas breathe, not so you feel obligated to fill it.

Questions

If anything in this spec is unclear, make your best judgment call and document your assumption. We'd rather see how you think through ambiguity than have you blocked waiting for clarification.